

Technische Dokumentation

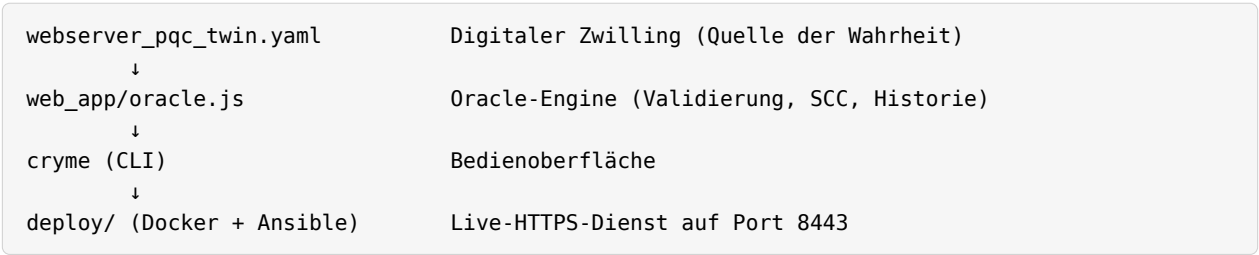
Quellcode, Architektur und Datenfluss

Masterprojekt · Hochschule RheinMain · September 2026

Diese Dokumentation erklärt den Quellcode, die Architektur und den Datenfluss von CRYME. Zielgruppe sind Gutachterinnen und Gutachter, die den Code nachvollziehen möchten, ohne jede Datei einzeln lesen zu müssen. Anwenderüberblick: `GUIDE.md` · Installation: `INSTALL.md` · Memgraph: **Memgraph-Anleitung**.

1. Was CRYME technisch macht

CRYME ist ein **Orchestrator** für geplante PQC-Migrationen. Es ersetzt keine TLS-Implementierung, sondern verbindet vier Schichten:



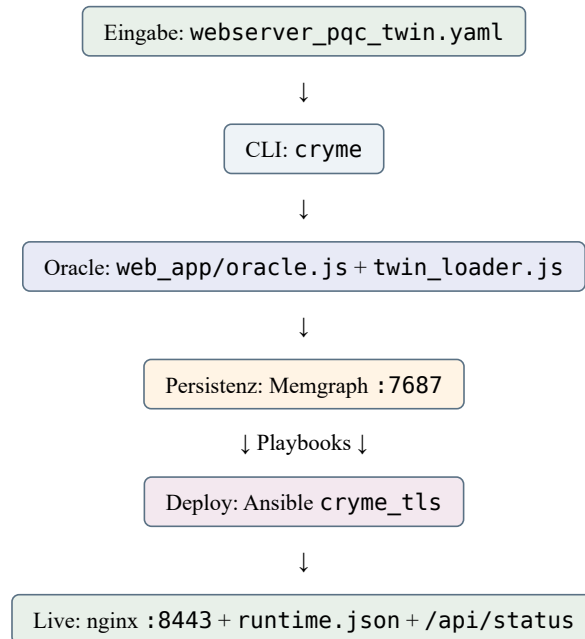
Kernidee: Jede Migration wird zuerst im Graphen (Memgraph) validiert. Erst bei Erfolg entsteht ein Ansible-Playbook und kann per `cryme deploy` auf den Live-Dienst übertragen werden.

2. Projektstruktur (Code-Karte)

Pfad	Sprache	Rolle
<code>cryme</code>	Node.js (CLI)	Einstiegspunkt — parst Argumente, ruft <code>oracle.js</code> und Shell-Skripte auf
<code>web_app/oracle.js</code>	JavaScript	Kernlogik: SCC, Migration, Replay, Playbook-Generierung
<code>web_app/twin_loader.js</code>	JavaScript	YAML → Memgraph laden (<code>cryme init</code>)
<code>web_app/server.js</code>	JavaScript	Optionale Web-UI (gleiche Oracle + Memgraph)
<code>web_app/check_db.js</code>	JavaScript	Bolt-Verbindungstest
<code>webserver_pqc_twin.yaml</code>	YAML	Digitaler Zwilling (Webserver + Browser)
<code>deploy/docker-compose.yml</code>	YAML	Memgraph, nginx, curl-client
<code>deploy/roles/cryme_tls/</code>	Ansible/Jinja2	TLS-Profil aus Migrationszustand ableiten
<code>deploy/verify_tls.sh</code>	Bash	TLS-Handshake + <code>/api/status</code> prüfen
<code>playbooks/</code>	Ansible YAML	Pro Migrationsschritt generierte Playbooks

Pfad	Sprache	Rolle
logs/	Text	Oracle-Ausgabe pro Schritt

3. Architekturdiagramm



Die Planung lebt in Memgraph. Der Live-Dienst ändert sich erst durch `cryme deploy`.

4. Modul: cryme (CLI)

Datei: cryme (ausführbares Node.js-Skript im Projektstamm)

4.1. Aufgaben

1. Kommandozeilenargumente parsen (show, migrate, deploy, verify, init)
2. Memgraph-Session öffnen und Funktionen aus `oracle.js` aufrufen
3. Für `deploy` und `verify`: externe Tools starten (ansible-playbook, `bash deploy/verify_tls.sh`)

4.2. Wichtige Befehle → Funktionen

CLI-Befehl	Oracle-Funktion / externes Tool
<code>cryme init</code>	<code>initDatabase() + resetLiveServiceState()</code>
<code>cryme migrate ...</code>	<code>migrateNodes()</code>
<code>cryme show state step=N</code>	<code>reconstructStateAtStep()</code>
<code>cryme show graph step=N</code>	<code>reconstructStateAtStep() + renderGraphAscii()</code>
<code>cryme show diff step=N</code>	<code>computeStepDiff()</code>
<code>cryme show step step=N</code>	<code>computeStepDiff() + renderStepShowAscii()</code>
<code>cryme show tree</code>	Cypher auf <code>MigrationStep + TRANSITION_T0</code>
<code>cryme show node</code>	Cypher auf <code>CryptoAsset / SecurityControl</code>

CLI-Befehl	Oracle-Funktion / externes Tool
cryme deploy step=N	getStepDeployInfo() → Ansible
cryme verify step=N	deploy/verify_tls.sh

4.3. Verbindung zu Memgraph

Die Verbindung wird in `web_app/oracle.js` definiert und von der CLI wiederverwendet:

```
const URI = "bolt://localhost:7687";
const driver = neo4j.driver(URI, neo4j.auth.basic("", ""));
```

Kein Benutzer, kein Passwort — Memgraph läuft lokal im Docker-Container.

5. Modul: `web_app/oracle.js` (Oracle-Engine)

Das ist das **technische Herzstück** (ca. 1300 Zeilen). Die CLI ist ein dünner Wrapper um die Exports am Dateiende.

5.1. Graph-Algorithmen (rein im Speicher)

Funktion	Algorithmus	Zweck
<code>computeSCCs()</code>	Tarjan	Strongly Connected Components — welche Knoten müssen gemeinsam migriert werden?
<code>computeTransitiveReduction</code>	Floyd–Warshall	Redundante Kanten entfernen (transitive Reduktion)

Diese Algorithmen laufen auf dem in den Speicher geladenen Graphen, nicht als Memgraph-Prozeduren.

5.2. Zustandsverwaltung

Funktion	Beschreibung
<code>loadStateFromDB()</code>	Liest aktuellen Graphen aus Memgraph (Knoten, Kanten, Historie)
<code>loadBaselineTopology()</code>	Statische Topologie aus YAML (ohne Laufzeit-Entdeckungen)
<code>reconstructStateAtStep(N)</code>	Event Replay: Zustand bei Schritt N durch Abspielen aller Events rekonstruieren
<code>getHeadStep()</code>	Aktueller HEAD-Zeiger (letzter erfolgreicher Schritt)
<code>updateHeadStep(N)</code>	Vorheriges HEAD löschen, <code>SystemMeta.head_step</code> und <code>MigrationStep.head</code> setzen

Event Sourcing: Der Zustand bei Schritt N wird nicht als Snapshot gespeichert, sondern aus Baseline + MigrationStep-Ereignissen 1..N **berechnet**. Details: `docs/GRAPH_VERSIONING.md`.

5.3. Migrationsablauf (`migrateNodes`)

1. Zielknoten auflösen (Name oder Memgraph-ID)
2. Cluster bilden (SCC über explizite + implizite + globale Kanten)
3. Pro Knoten Zielalgorithmus prüfen (PQCVariant aus YAML)
4. Oracle-Validierung (`checkOracleValidation`):
 - Bereits migriert? → redundant
 - Temporal constraints (`not_before`)?
 - Strukturelle Abhängigkeiten?

5. Bei Fehler: `discovered_edge` speichern (Oracle lernt neue Kante)
6. Bei Erfolg:
 - Knoten in Memgraph auf `status=migrated` setzen
 - MigrationStep-Event anlegen
 - HEAD aktualisieren
 - Ansible-Playbook generieren
 - Log schreiben

Fehlgeschlagene Schritte erzeugen trotzdem einen MigrationStep (Status failed oder aborted), damit der Historienbaum vollständig bleibt. HEAD wandert bei Fehlern **nicht**.

5.4. Oracle-Validierung (`checkOracleValidation`)

Prüft unter anderem:

- **Co-Migration:** Knoten in derselben SCC müssen zusammen migriert werden
- **Implizite Abhängigkeiten:** SecurityControls hängen von CryptoAssets ab
- **Globale Abhängigkeiten:** Server-KEX ↔ Browser-KEX (Demo-Schritt 1 schlägt hier fehl)
- **Temporal:** `not_before`-Constraints zwischen Komponenten

5.5. Deploy-Vorbereitung

Funktion	Beschreibung
<code>getStepDeployInfo(N)</code>	Rekonstruiert Zustand bei Schritt N, schreibt <code>deploy/vars/step_N.json</code>
<code>buildDeployVarsFromState()</code>	Mappt Knoten → Algorithmen für Ansible
<code>generatePlaybookContent()</code>	Schreibt Playbook mit <code>include_role: cryme_tls</code>
<code>resetLiveServiceState()</code>	Setzt <code>runtime.json</code> / <code>data.json</code> auf Baseline und lädt nginx neu (cryme init)

`cryme deploy` liest Memgraph (per Replay). Es schreibt **keine** neuen Graph-Ereignisse.

6. Modul: `web_app/twin_loader.js`

Wird von `initDatabase()` bei `cryme init` aufgerufen.

6.1. Ablauf

1. `initDatabase()` leert Memgraph (MATCH (n) DETACH DELETE n)
2. `populateMemgraphWithTwin()` liest `webserver_pqc_twin.yaml`:
 - Component-Knoten anlegen
 - CryptoAsset + PQCVariant-Knoten anlegen
 - SecurityControl-Knoten anlegen
3. Kanten anlegen: HAS_ASSET, HAS_CONTROL, HAS_VARIANT, EXPLICIT_DEPENDENCY, IMPLICIT_DEPENDENCY, GLOBAL_DEPENDENCY, TEMPORAL_CONSTRAINT

SystemMeta und der initiale MigrationStep (Schritt 0, status: init) entstehen **nicht** im Loader. Die erste `cryme migrate` erzeugt sie per `MERGE (init:MigrationStep {step: 0}); updateHeadStep()` legt SystemMeta an.

Quelle: `webserver_pqc_twin.yaml`

7. Modul: Ansible-Rolle `cryme_tls`

Pfad: `deploy/roles/cryme_tls/`

7.1. Eingabe (via `cryme deploy step=N`)

```
{
  "migration_step": 2,
  "migrated_nodes": [
    "Webserver_Classic.KeyExchange_ECDHE",
    "Client_Browser.KeyExchange_ECDHE"
  ],
  "target_algorithms": {
    "Webserver_Classic.KeyExchange_ECDHE": "X25519_MLKEM768",
    "Client_Browser.KeyExchange_ECDHE": "X25519_MLKEM768"
  }
}
```

7.2. Ableitung des TLS-Profiles (`tasks/main.yml`)

Aus `target_algorithms` werden abgeleitet:

Bedingung	TLS-Profil	nginx-Zertifikat	Protokolle
Alles classic	<code>classic-rsa-ecdhe</code>	RSA-2048	TLS 1.2 + 1.3
Hybrid KEX	<code>hybrid-kex-classic-cert</code>	RSA-2048	TLS 1.2 + 1.3
ML-DSA Cert	<code>hybrid-kex-mldsa-cert</code>	ECDSA (Stand-in)	TLS 1.2 + 1.3
Nur TLS 1.3	<code>tls13-only</code>	je nach Cert	nur TLS 1.3

7.3. Generierte Dateien

Datei	Inhalt
<code>deploy/nginx/live/tls.conf</code>	nginx TLS-Konfiguration
<code>deploy/state/runtime.json</code>	API-Zustand für <code>/api/status</code>
<code>deploy/state/data.json</code>	API-Zustand für <code>/api/data</code>
<code>deploy/client/expect.env</code>	Erwartungen für curl-client (Browser-Simulation)

nginx liest `runtime.json` statisch (Volume-Mount). Die API-Antwort kommt direkt aus dieser Datei — kein separater API-Container nötig.

8. Live-Dienst (Port 8443)

8.1. Docker-Stack

Container	Port	Funktion
<code>cryme-memgraph</code>	7687	Graph-Datenbank
<code>cryme-memgraph-lab</code>	3000	Web-GUI
<code>cryme-nginx-classic</code>	8443→443	HTTPS + statische API-Dateien
<code>cryme-curl-client</code>	—	Simulierter Browser

8.2. API-Endpunkte

nginx serviert Dateien aus `deploy/state/`:

URL	Datei	Inhalt
<code>/api/status</code>	<code>runtime.json</code>	<code>migration_step, profile, algorithms</code>
<code>/api/data</code>	<code>data.json</code>	Beispiel-Payload
<code>/health</code>	<code>inline</code>	Health-Check + Header X-Cryme-Tls-Profile

8.3. Zwei Sichten auf denselben Zustand

```
cryme show state step=2
# Planungszustand (Memgraph-Replay)

curl -sk https://127.0.0.1:8443/api/status
# Laufzeitzustand (nginx)
```

Nach `cryme deploy step=2` müssen beide übereinstimmen.

9. Verifikation (`deploy/verify_tls.sh`)

Prüft unabhängig von CRYME:

1. **openssl s_client** — Zertifikat, Protokoll, Cipher
2. **curl vom Host** — `/health`
3. **curl aus curl-client-Container** — simuliert Client_Browser
4. **nginx-Header** — X-Cryme-Tls-Profile
5. **API-Status** — `curl -sk .../api/status`, vergleicht `migration_step` mit dem erwarteten Schritt

10. Datenmodell (Kurzform)

10.1. Knotentypen in Memgraph

Label	Beispiel-ID	Wichtige Properties
Component	<code>Webserver_Classic</code>	<code>phase</code>
CryptoAsset	<code>Webserver_Classic.KeyExchange_ECDHE</code>	<code>status, algorithm, active_algorithm</code>
SecurityControl	<code>Webserver_Classic.TLS_1.2/_1.3_Communication</code>	<code>status, active_algorithm</code>
PQCVariant	<code>...mlkem768</code>	<code>algorithm, security_level</code>
MigrationStep	<code>step=2</code>	<code>status, cluster, variants, head</code>
SystemMeta	<code>cryme</code>	<code>head_step</code>

10.2. Kantentypen

Relation	Bedeutung
<code>HAS_ASSET</code>	<code>Component → CryptoAsset</code>
<code>HAS_CONTROL</code>	<code>Component → SecurityControl</code>

Relation	Bedeutung
HAS_VARIANT	CryptoAsset → PQCVariant
EXPLICIT_DEPENDENCY	Deklarierte funktionale Abhängigkeit
IMPLICIT_DEPENDENCY	Implizite Abhängigkeit (inkl. zur Laufzeit entdeckte)
GLOBAL_DEPENDENCY	Komponentenübergreifend (Server ↔ Browser)
TEMPORAL_CONSTRAINT	Phasenordnung (<code>not_before</code>)
TRANSITION_TO	Migrationshistorie (Baum, nicht Abhängigkeitsgraph)

Details: docs/DOMAIN_MODEL.md

11. Typischer Demo-Ablauf (technisch)

```

cryme init
  → twin_loader.js lädt YAML in Memgraph
  → resetLiveServiceState() setzt runtime.json auf step=0

cryme migrate id=Webserver_Classic.KeyExchange_ECDHE X25519_MLKEM768
  → migrateNodes() findet GLOBAL_DEPENDENCY zu Client_Browser
  → Oracle: FAIL (Schritt 1), discovered_edge gespeichert

cryme migrate id=Webserver_Classic.KeyExchange_ECDHE X25519_MLKEM768
  → SCC enthält jetzt beide KEX-Knoten
  → Oracle: SUCCESS (Schritt 2), Playbook generiert

cryme deploy step=2
  → getStepDeployInfo(2) → deploy/vars/step_2.json
  → ansible-playbook apply_tls.yml
  → nginx TLS-Profil + runtime.json aktualisiert

cryme verify step=2
  → verify_tls.sh prüft TLS + migration_step=2 in /api/status

```

Die zugehörigen Cypher-Queries stehen in der **Memgraph-Anleitung**, Abschnitt Demo-Schritte 0–4.

12. Code in dieser Reihenfolge lesen

Wenn Sie die Implementierung prüfen, entspricht diese Reihenfolge dem Laufzeitpfad:

1. `webserver_pqc_twin.yaml` — was der Zwilling deklariert
2. `web_app/twin_loader.js` — wie YAML zu Graphknoten wird
3. `web_app/oracle.js` — `loadStateFromDB`, `checkOracleValidation`, `migrateNodes`, `reconstructStateAtStep`
4. `cryme` — nur CLI-Dispatch
5. `deploy/roles/cryme_tls/tasks/main.yml` — wie ein geplanter Schritt zum TLS-Profil wird
6. `deploy/verify_tls.sh` — unabhängiger Nachweis auf Port 8443

`web_app/server.js` und `web_app/public/app.js` sind eine optionale UI auf derselben Memgraph-Datenbank. Der Demo-Pfad für die Bewertung ist die CLI.

13. Bekannte technische Limitierungen (Phase B)

Bereich	Ist-Zustand	Geplant (Phase C)
PQC on-the-wire	ML-KEM/ML-DSA als Namen ; OpenSSL nutzt klassisches TLS	OQS-nginx für echte PQC-Handshakes
Zertifikat ML-DSA	ECDSA als Stand-in	Echtes ML-DSA-Zertifikat
Szenarien	Webserver live, Automotive nur in YAML	Weitere Digital Twins

14. Weiterführende Dokumentation

Dokument	Inhalt
docs/MEMGRAPH_ANLEITUNG.md	Memgraph bedienen, Beispielqueries
docs/GRAPH_VERSIONING.md	Event Sourcing, HEAD, Replay
docs/migration_explanation.md	Oracle-SCC-Verhalten im Detail
docs/DOMAIN_MODEL.md	ER-Diagramm, Namensregeln
docs/TLS_ALGORITHMS.md	TLS-Profile-Matrix
docs/KI_NUTZUNG.md	Dokumentation der KI-Nutzung